Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Ecole superieure d'informatique
ESI-SBA

Spécialité : Intelligence Artificielle & science du données

RAPPORT DE PROJET

Meetly

Système Intelligent de Reconnaissance et de Résumé Automatique de Réunions en Dialecte Algérien

Fine-tuning de Whisper Large-V3 avec LoRA · Diarisation · Résumé Automatique

Réalisé par :
- Kambouz Nouha
- Amari Soumia
- Mohammed ousaid Ahlem
- Merzouk Imane Nabahat
- Berrahou meriem

Encadré par : ***Mr .SERHANE Oussama***

Année universitaire : 2025–2026

Résumé

Résumé en Français

Ce projet présente Meetly, un système complet de traitement automatique de la parole dialectale algérienne (Darja), conçu comme une plateforme intelligente d'analyse et de compréhension de réunions. Le système vise principalement la génération automatique de résumés structurés de réunions, tout en s'appuyant sur des capacités avancées de traitement de la parole.
Il intègre trois composants principaux : (1) un module de Reconnaissance Automatique de la Parole (ASR) basé sur le fine-tuning du modèle whisper-large-v3 d'OpenAI via la technique PEFT/LoRA, chargé de transformer l'audio en texte exploitable, (2) un module de Diarisation permettant d'identifier les locuteurs et de structurer les interventions, et (3) un module de Résumé Automatique, cœur du système, générant une synthèse cohérente, organisée et exploitable des réunions.
L'ensemble est déployé via une application web interactive permettant la consultation des transcriptions, des intervenants et des résumés générés.
Le travail a nécessité la construction d'un dataset original à partir de sources hétérogènes, le développement de plusieurs versions successives du modèle ASR, ainsi que la résolution de défis techniques majeurs incluant la corruption de données, le sur-apprentissage et les contraintes mémoire GPU. Le modèle final a été amélioré progressivement grâce à un pipeline de nettoyage de données et d'augmentation audio, afin de garantir une base fiable pour la génération de résumés de réunions de haute qualité..

Abstract (English)

This project presents DarjaVoice, a comprehensive intelligent meeting assistance system for the Algerian Darja dialect, designed with a primary focus on automatic generation of structured and meaningful meeting summaries from spoken content. The system integrates fine-tuning of OpenAI's whisper-large-v3 model using PEFT/LoRA techniques for transcription, a speaker diarization module for structuring and attributing spoken segments, and a central automatic summarization module responsible for producing concise, coherent, and actionable meeting summaries. All components are integrated into an interactive web application. A custom dataset was constructed from heterogeneous sources, and three progressive model versions were developed to address challenges such as data corruption,

overfitting, and GPU memory constraints, ultimately improving the quality of generated summaries.

Mots-clés : Résumé Automatique, Reconnaissance Automatique de la Parole, Darja Algérienne, Whisper, LoRA, PEFT, Diarisation, Code-Switching, NLP, Deep Learning.

Table des matières

Résumé **1**

1 Introduction Générale **8**
1.1 Contexte et Motivation 8
1.2 Problématique 8
1.3 Objectifs du Projet 9
1.3.1 Objectifs Principaux 9
1.3.2 Objectifs Secondaires 9

2 État de l'Art **10**
2.1 Reconnaissance Automatique de la Parole (ASR) 10
2.1.1 Évolution des Architectures ASR 10
2.1.2 OpenAI Whisper : Présentation Générale 10
2.2 Fine-tuning Efficace en Paramètres (PEFT) 11
2.2.1 Le Problème du Fine-tuning Complet 11
2.2.2 Low-Rank Adaptation (LoRA) 11
2.3 Diarisation des Locuteurs 12
2.4 Résumé Automatique de texte 12
2.5 Langues à Faibles Ressources et Code-Switching 13

3 Construction du Dataset **14**
3.1 Analyse des Besoins en Données 14
3.2 Phase 1 : Dataset de Base (Hugging Face) 14
3.2.1 Source Initiale : ayoubkirouane/Algerian-Darija 14
3.2.2 Limitation du Dataset Initial 14
3.3 Phase 2 : Extension par Collecte YouTube 15
3.3.1 Stratégie de Collecte 15
3.3.2 Critères de Sélection des Vidéos 15
3.4 Phase 3 : Fusion et Construction du Dataset Combiné 15
3.4.1 Processus de Fusion 15
3.4.2 Statistiques du Dataset Combiné 16
3.5 Phase 4 : Augmentation Audio Offline 16
3.5.1 Techniques d'Augmentation 16
3.5.2 Résultat de l'Augmentation 17
3.6 Nettoyage Algorithmique des Données Corrompues 17
3.6.1 Découverte des Anomalies 17
3.6.2 Pipeline de Purge Algorithmique 18

4 Architecture du Modèle Whisper et Fine-tuning **19**
4.1 Architecture Détaillée de Whisper Large-V3 . . . 19
4.1.1 Vue d'Ensemble de l'Architecture . . . 19
4.1.2 L'Encodeur Audio . . . 20
4.1.3 Le Décodeur Texte . . . 21

5 Développement Progressif du Modèle ASR — Trois Versions **22**
5.1 Version 1 : Baseline Architecture (whisper-medium) . . . 22
5.1.1 Configuration et Paramètres . . . 22
5.1.2 Résultats et Analyse — Effondrement du Baseline . . . 22
5.1.3 Diagnostic et Remèdes Apportés . . . 23
5.2 Version 2 : Régularisation et Correction des Métriques . . . 23
5.2.1 Améliorations Architecturales . . . 23
5.2.2 Résultats et Analyse de la Version 2 . . . 24
5.3 Version 3 : Pipeline "Invincible" — whisper-large-v3 + LoRA . . . 25
5.3.1 Vue d'Ensemble des Améliorations . . . 25
5.3.2 Configuration LoRA Étendue . . . 25
5.3.3 SpecAugment — Augmentation dans l'Espace Spectral . . . 26
5.3.4 Résolution du Bug de Fuite de Données . . . 26
5.3.5 Contournement du Blocage PyTorch 2.5 . . . 26
5.3.6 Hyperparamètres Finaux — Version 3 . . . 27
5.3.7 Résultats Finaux — Version 3 . . . 27
5.3.8 Analyse de Convergence . . . 29
5.4 Comparaison des Trois Versions . . . 29

6 Diagrammes UML et Architecture Système **30**
6.1 Diagramme de Cas d'Utilisation (Use Case) . . . 30
6.2 Diagramme d'Activité . . . 31
6.3 Diagramme de Classes . . . 32
6.4 Diagramme de Composants . . . 32
6.5 Diagramme de Déploiement . . . 33
6.6 Diagramme de Flux de Données — Pipeline Complet . . . 33

7 Diarisation, Résumé et Application Web **34**
7.1 Module de Diarisation des Locuteurs . . . 34
7.1.1 Problème de la Diarisation . . . 34
7.1.2 Architecture du Module de Diarisation . . . 34
7.1.3 Fusion Transcription + Diarisation . . . 35
7.2 Module de Résumé Automatique . . . 35
7.2.1 Approche Retenue . . . 35
7.3 Application Web Interactive . . . 36
7.3.1 Architecture Technique . . . 36
7.3.2 Stack Technique . . . 38

8 Résultats, Analyse et Perspectives **39**
8.1 Résultats Globaux et Comparatifs . . . 39
8.1.1 Métriques de Performance ASR . . . 39
8.2 Analyse des Erreurs Résiduelles . . . 39
8.3 Perspectives et Travaux Futurs . . . 39

8.3.1 Court Terme 39
8.3.2 Moyen Terme 39
8.3.3 Long Terme 40

Conclusion Générale **41**

9 Glossaire Technique **43**

Table des figures

2.1 Architecture Low-Rank Adaptation (LoRA) 12
2.2 Architecture Qwen 13

3.1 Data augmentation architecture 17

4.1 Architecture Globale du Système DarjaVoice 20

5.1 Courbes de perte et WER — Version 1 (Step 1000) : effondrement visible du WER à ∼106.6% 23
5.2 Courbes de convergence — Version 2 (Step 200) : début de stabilisation de l'évaluation 24
5.3 Logs d'entraînement détaillés 27
5.4 Results sur le test 28
5.5 Diagramme de Séquence — Processus d'Entraînement 28

6.1 Diagramme de Cas d'Utilisation — Système DarjaVoice 30
6.2 Diagramme d'Activité — Flux Principal DarjaVoice 31
6.3 Diagramme de Classes — Architecture Orientée Objet 32
6.4 Diagramme de Composants — Architecture Logicielle 32
6.5 Diagramme de Déploiement — Infrastructure Cible 33
6.6 Data Flow Diagram — Pipeline Complet DarjaVoice 33

7.1 Interface Web — Dashboard 36
7.2 Interface Web — Record 36
7.3 Interface Web — Meeting history 37
7.4 Interface Web — Transcript 37
7.5 Interface Web — summury 38

Liste des tableaux

2.1 Évolution des approches ASR 10
2.2 Variantes du modèle Whisper d'OpenAI 11

3.1 Caractéristiques du dataset de base Hugging Face 14
3.2 Statistiques du dataset combiné avant augmentation 16
3.3 Techniques d'augmentation audio appliquées 16

5.1 Configuration de la Version 1 (Baseline) 22
5.2 Logs d'entraînement détaillés — Version 1 24
5.3 Hyperparamètres finaux de la Version 3 (whisper-large-v3 + LoRA) . . 27
5.4 Bilan comparatif des trois versions du modèle ASR 29

7.1 Stack technique de l'application web 38

8.1 Comparaison des performances des trois versions ASR 39

Chapitre 1

Introduction Générale

1.1 Contexte et Motivation

L'Algérie se distingue par un paysage linguistique d'une complexité exceptionnelle. La langue vernaculaire quotidienne, le Darja algérien, constitue un système linguistique hybride et en constante évolution qui défie les approches classiques du traitement automatique du langage naturel (TALN). Contrairement à l'Arabe Standard Moderne (ASM), le Darja ne possède ni académie normative, ni orthographe codifiée, ni corpus numérique standardisé de grande envergure.

Dans ce contexte, le domaine des réunions professionnelles et académiques représente un cas d'usage particulièrement pertinent. En Algérie, les réunions se déroulent quasi-exclusivement en Darja, avec des insertions fréquentes de vocabulaire français et, dans une moindre mesure, anglais. L'absence d'outils adaptés ne constitue pas seulement un problème de transcription, mais surtout une limitation majeure dans la capacité à produire automatiquement des résumés structurés et exploitables des réunions, indispensables pour la prise de décision et le suivi des actions.

Meetly répond directement à cette lacune en proposant un système end-to-end capable de :

1. Transcrire avec précision la parole en Darja algérien ;
2. Identifier les intervenants (diarisation) ;
3. Générer un résumé structuré de la réunion ;
4. Présenter ces résultats dans une interface web accessible et interactive.

1.2 Problématique

Problématique Centrale

Comment construire un système intelligent capable de générer des résumés de réunions fiables, structurés et exploitables à partir du Darja algérien — une langue à ressources quasi-nulles, fortement code-switchée, dépourvue d'orthographe normalisée — en s'appuyant sur un modèle pré-entraîné généraliste, tout en surmontant les contraintes

de données, matérielles et algorithmiques liées à la transcription, à la structuration des échanges et à la génération automatique de résumés ?

1.3 Objectifs du Projet

1.3.1 Objectifs Principaux

—— Construire un dataset ASR pour le Darja algérien à partir de sources hétérogènes ;
—— Fine-tuner whisper-large-v3 via LoRA pour la transcription du Darja ;
—— Intégrer un système de diarisation des locuteurs afin de structurer les échanges ;
—— Développer un système de résumé automatique de réunions, objectif central du projet, capable de générer des synthèses structurées, cohérentes et exploitables ;
—— Déployer l'ensemble sous forme d'une application web interactive orientée résumé de réunions.

1.3.2 Objectifs Secondaires

➔ Analyser l'impact des différentes stratégies de fine-tuning sur les métriques WER/-CER ;
➔ Identifier et corriger les corruptions de données via des techniques algorithmiques ;
➔ Étudier l'amélioration de la qualité des résumés de réunions en fonction des performances du pipeline ASR et diarisation ;
➔ Documenter les obstacles techniques et les solutions apportées et Proposer une architecture de déploiement optimisée pour la génération fiable de résumés de réunions ;

Chapitre 2

État de l'Art

2.1 Reconnaissance Automatique de la Parole (ASR)

2.1.1 Évolution des Architectures ASR

Le domaine de l'ASR a connu une révolution paradigmatique avec l'adoption des architectures Transformer. Les modèles classiques à base de GMM-HMM (Gaussian Mixture Models - Hidden Markov Models) ont progressivement cédé la place à des approches neuronales de bout-en-bout, culminant avec l'émergence des grands modèles de langage audio pré-entraînés.

TABLE 2.1 – Évolution des approches ASR

Époque	**Architecture**	**Paramètres**	**Caractéristique**
2000–2012	GMM-HMM	<1M	Modélisation phonème par phonème
2012–2018	RNN/LSTM	1–100M	Mémoire séquentielle, CTC
2018–2020	Transformer (Wav2Vec)	100M–1B	Représentations contrastives
2020–2022	Conformer	100M–600M	CNN + Transformer hybride
2022–2024	Whisper (OpenAI)	39M–1.55B	Supervisé, multilingue, robuste

2.1.2 OpenAI Whisper : Présentation Générale

Whisper [1] est un modèle séquence-à-séquence (Seq2Seq) entraîné de manière entièrement supervisée sur **680 000 heures** d'audio multilingue collecté sur Internet. Son architecture encode les caractéristiques acoustiques via un encodeur Transformer et génère la transcription de manière autoregressive via un décodeur Transformer. Il existe en plusieurs variantes :

TABLE 2.2 – Variantes du modèle Whisper d'OpenAI

Modèle	Paramètres	Couches Enc.	Couches Déc.	Têtes d'Att.
tiny	39M	4	4	6
base	74M	6	6	8
small	244M	12	12	12
medium	769M	24	24	16
large-v3	**1 550M**	**32**	**32**	**20**

2.2 Fine-tuning Efficace en Paramètres (PEFT)

2.2.1 Le Problème du Fine-tuning Complet

Le fine-tuning complet (*full fine-tuning*) d'un modèle à 1,55 milliard de paramètres requiert :

— ∼24 GB de VRAM GPU pour les gradients et les états de l'optimiseur ;
— Un risque élevé d'*oubli catastrophique* (*catastrophic forgetting*) où le modèle écrase ses connaissances générales pour s'adapter au dialecte cible.

2.2.2 Low-Rank Adaptation (LoRA)

LoRA [2] est une technique PEFT qui gèle les poids originaux $W_0 \in \mathbb{R}^{d\times k}$ et injecte une paire de matrices de bas rang (A, B) :

$$h = W_0 x + \Delta W x = W_0 x + BAx \tag{2.1}$$

où $A \in \mathbb{R}^{r\times k}$, $B \in \mathbb{R}^{d\times r}$, et $r \ll \min(d, k)$ est le rang LoRA. Le nombre de paramètres entraînables passe de $d \times k$ à $r(d + k)$, soit une réduction de 99% pour $r = 32$.

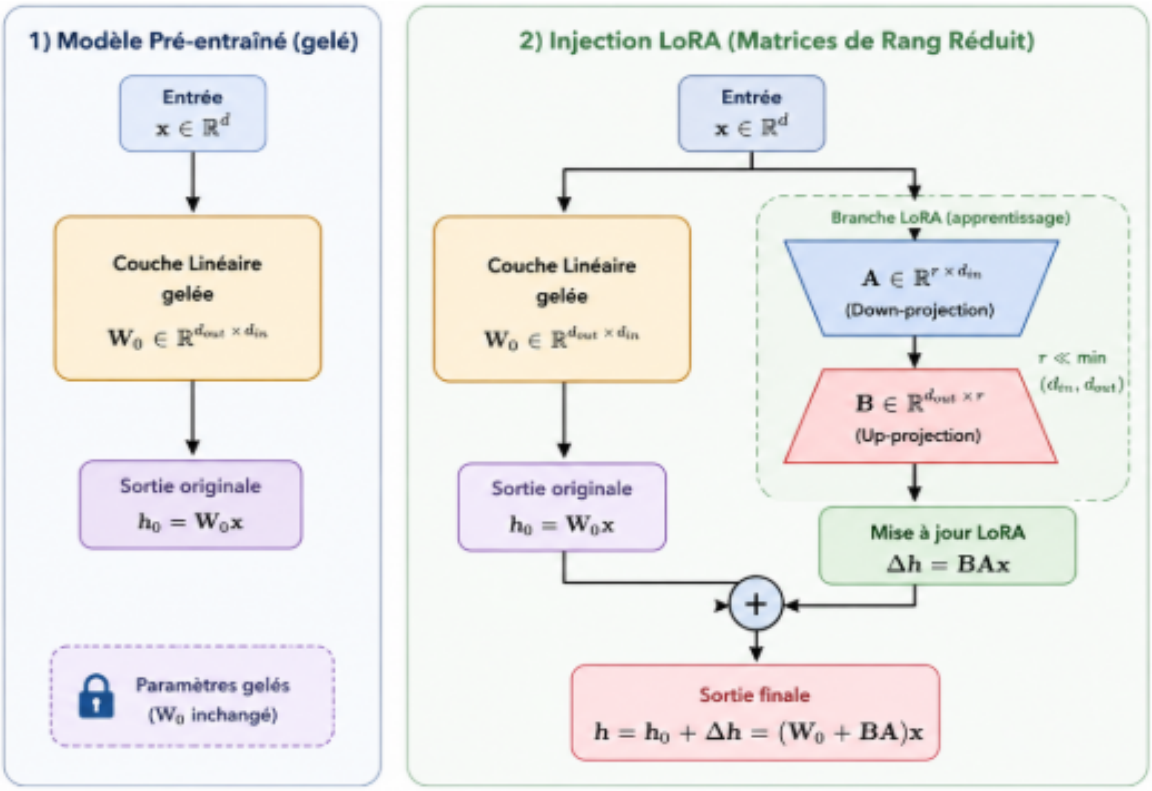


FIGURE 2.1 – Architecture Low-Rank Adaptation (LoRA)

2.3 Diarisation des Locuteurs

La diarisation (*speaker diarization*) résout le problème "qui parle quand ?" en segmentant un flux audio en segments homogènes par locuteur. Les approches modernes utilisent des modèles de *speaker embeddings* (x-vectors, d-vectors) combinés à des algorithmes de clustering.

2.4 Résumé Automatique de texte

Pour la génération de résumés à partir des transcriptions, deux approches sont envisageables :

— **Extractive** : sélection des phrases les plus représentatives (ex. TextRank, BERTSum) ;
— **Abstractive** : génération de nouvelles phrases via un LLM (ex. BART, T5, GPT).

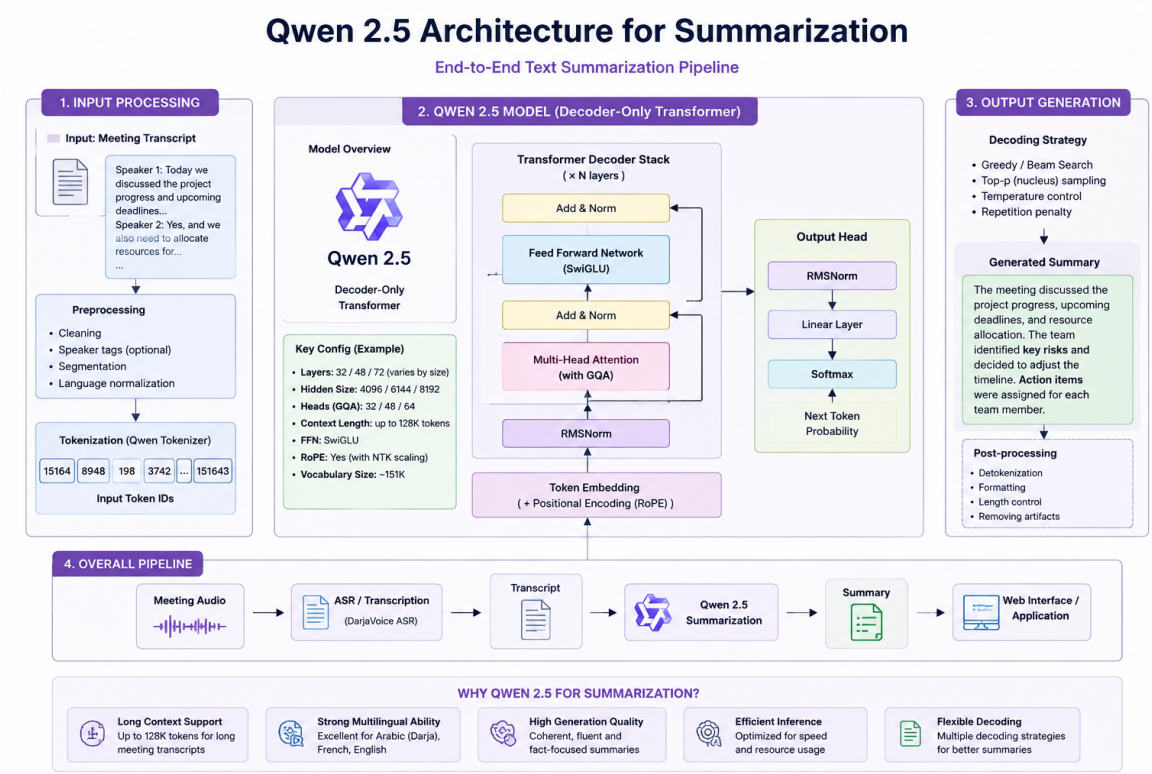


FIGURE 2.2 – Architecture Qwen

2.5 Langues à Faibles Ressources et Code-Switching

Le défi fondamental des langues à faibles ressources (*low-resource languages*) réside dans la scarcité des données annotées. Le Darja algérien présente en outre le phénomène de **code-switching**, où l'alternance de codes linguistiques au sein d'un même énoncé crée des ambiguïtés phonétiques et lexicales difficiles à modéliser.

Chapitre 3

Construction du Dataset

3.1 Analyse des Besoins en Données

Un système ASR de qualité nécessite typiquement des milliers d'heures de données audio annotées. Pour le Darja algérien, les ressources disponibles publiquement sont extrêmement limitées. Notre stratégie de construction du dataset s'est articulée en deux phases complémentaires.

3.2 Phase 1 : Dataset de Base (Hugging Face)

3.2.1 Source Initiale : ayoubkirouane/Algerian-Darija

Nous avons commencé par le dataset `ayoubkirouane/Algerian-Darija` disponible sur la plateforme Hugging Face. Ce dataset constitue une base annotée de clips audio courts en Darja algérien avec leurs transcriptions correspondantes.

TABLE 3.1 – Caractéristiques du dataset de base Hugging Face

Caractéristique	**Valeur**
Source	Hugging Face Hub
Identifiant	`ayoubkirouane/Algerian-Darija`
Format audio	WAV, 16 kHz, mono
Transcription	Texte en Arabe (Darja algérien)
Taille initiale	∼1 000–2 000 clips
Durée moyenne des clips	5–15 secondes

3.2.2 Limitation du Dataset Initial

Le dataset initial s'est révélé insuffisant en volume pour entraîner un modèle robuste. Les modèles de deep learning pour l'ASR nécessitent généralement au minimum plusieurs milliers d'heures d'audio ; notre dataset de base représentait un régime de très faible ressource nécessitant une augmentation significative.

3.3 Phase 2 : Extension par Collecte YouTube

3.3.1 Stratégie de Collecte

Pour enrichir le dataset, nous avons adopté une approche systématique de collecte à partir de vidéos YouTube en Darja algérien :

Algorithm 1 Pipeline de Collecte et Prétraitement YouTube

1: Identifier des vidéos YouTube en Darja algérien (contenus authentiques : vlogs, interviews, émissions)
2: Télécharger la piste audio via `yt-dlp`
3: Convertir en WAV 16 kHz mono via `ffmpeg`
4: Segmenter chaque audio en clips de **20 secondes** avec overlap minimal
5: Transcrire manuellement ou semi-automatiquement chaque clip en Darja
6: Valider la qualité de la transcription
7: Intégrer au dataset sous format Parquet (`audio, text, duration`)

3.3.2 Critères de Sélection des Vidéos

— Contenu authentique en Darja algérien (dialecte naturel, non scripté) ;
— Qualité audio acceptable (bruit ambiant modéré toléré) ;
— Diversité régionale (Alger, Oran, Constantine, Annaba...) ;
— Diversité des locuteurs (âge, genre, contexte social) ;
— Représentation des phénomènes de code-switching (Arabe/Français/Anglais).

3.4 Phase 3 : Fusion et Construction du Dataset Combiné

3.4.1 Processus de Fusion

Les deux sources de données ont été fusionnées dans un dataset unifié au format Parquet :

```python
import pandas as pd

# Charger les deux sources
df_huggingface = pd.read_parquet("algerian_darija_hf.parquet")
df_youtube     = pd.read_parquet("youtube_collected.parquet")

# Normalisation des colonnes
df_combined = pd.concat([df_huggingface, df_youtube], ignore_index=True)
df_combined.columns = ["audio", "text"]

# Sauvegarde du dataset combin
df_combined.to_parquet("combined_dataset.parquet", index=False)
print(f"Dataset combin  : {len(df_combined)} clips")
```

Listing 3.1 – Fusion des datasets en un dataset combiné

3.4.2 Statistiques du Dataset Combiné

TABLE 3.2 – Statistiques du dataset combiné avant augmentation

Métrique	Valeur
Nombre total de clips	∼4 000
Durée totale estimée	∼15–20 heures
Durée moyenne d'un clip	∼15–20 secondes
Format audio	WAV, 16 kHz, mono
Format texte	Arabe (Darja, code-switched)
Rapport train/test	90% / 10%

3.5 Phase 4 : Augmentation Audio Offline

Pour tripler artificiellement le volume du dataset, nous avons développé un pipeline d'augmentation audio utilisant la bibliothèque `audiomentations` :

```
from audiomentations import Compose, AddGaussianNoise, TimeStretch,
    PitchShift

augmenter = Compose([
    AddGaussianNoise(min_amplitude=0.001, max_amplitude=0.015, p
        =0.5),
    TimeStretch(min_rate=0.8, max_rate=1.2, p=0.5),
    PitchShift(min_semitones=-2, max_semitones=2, p=0.5)
])

# Pour chaque clip original : g n ration de 2 versions
    augment es
for index, row in df.iterrows():
    samples, sr = librosa.load(audio_path, sr=16000)
    for aug_num in range(1, 3):
        aug_samples = augmenter(samples=samples, sample_rate=sr)
        sf.write(new_aug_path, aug_samples, sr)
```

Listing 3.2 – Pipeline d'augmentation audio (augment_audio.py)

3.5.1 Techniques d'Augmentation

TABLE 3.3 – Techniques d'augmentation audio appliquées

Technique	Description	Objectif
Bruit Gaussien	Ajout de bruit aléatoire ($\sigma \in [0.001, 0.015]$), $p = 0.5$	Robustesse aux environnements bruités
Étirement Temporel	Modification de la vitesse ($\times 0.8$ à $\times 1.2$), $p = 0.5$	Invariance à la vitesse de parole
Décalage en Hauteur	Décalage de -2 à $+2$ demi-tons, $p = 0.5$	Robustesse aux variations de timbre vocal

3.5.2 Résultat de l'Augmentation

Le dataset est passé de ∼4 000 clips originaux à **∼12 000 clips augmentés**, soit un facteur de multiplication par 3.

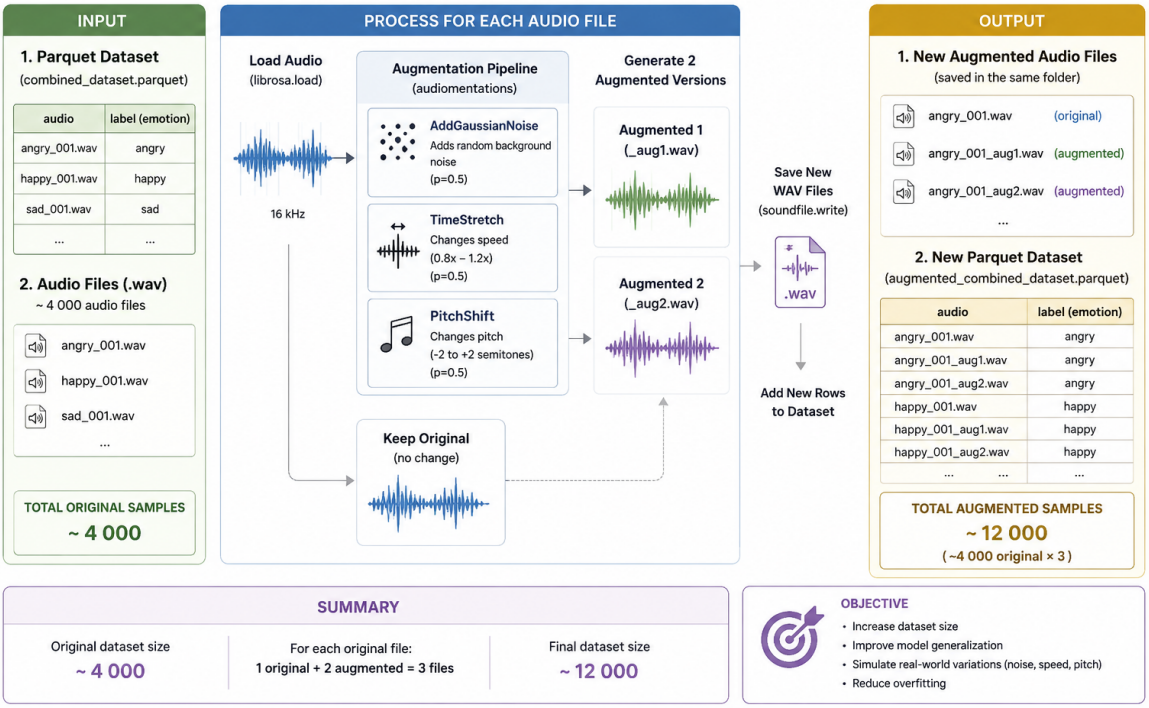


FIGURE 3.1 – Data augmentation architecture

3.6 Nettoyage Algorithmique des Données Corrompues

3.6.1 Découverte des Anomalies

L'analyse forensique des logs d'évaluation a révélé deux catégories majeures de corruption :

Anomalie 1 : Boucles de Hallucination dans les Labels

Anomalie Critique — Boucle d'Hallucination

Fichier : `ds1_clip_00117.wav`
WER résultant : 90.17%
Diagnostic : Le label est corrompu par une boucle de pseudo-labeling. Le modèle était pénalisé pour avoir correctement transcrit l'audio.

Anomalie 2 : Désalignement Total Audio-Texte

Anomalie Critique — Désalignement Audio/Texte

Fichier : `clip_000552.wav`
WER résultant : 100.0%
Diagnostic : Le fichier audio est physiquement désaligné avec son label dans le DataFrame de la phase d'agrégation.

3.6.2 Pipeline de Purge Algorithmique

```python
def has_repetition_loops(text, n=3, max_repeats=2):
    """D tecte les boucles de r p tition dans les labels
       corrompus."""
    words = str(text).split()
    if len(words) < n * (max_repeats + 1):
        return False
    for i in range(len(words) - n * max_repeats):
        ngram = tuple(words[i:i+n])
        repeats = 1
        for j in range(i+n, len(words) - n + 1, n):
            if tuple(words[j:j+n]) == ngram:
                repeats += 1
                if repeats > max_repeats:
                    return True
            else:
                break
    return False

# Filtrage des labels corrompus
df_clean = df[~df["text"].apply(has_repetition_loops)]

# Filtrage par ratio dur e/longueur de texte
df_clean = df_clean[
    df_clean.apply(lambda r:
        librosa.get_duration(filename=r["audio"]) /
        max(len(r["text"].split()), 1) > 0.5,
    axis=1)
]
```

Listing 3.3 – Détecteur de boucles de répétition n-gram

Chapitre 4

Architecture du Modèle Whisper et Fine-tuning

4.1 Architecture Détaillée de Whisper Large-V3

4.1.1 Vue d'Ensemble de l'Architecture

Whisper est une architecture Transformer séquence-à-séquence. Son pipeline de traitement s'articule en cinq étapes fondamentales :

1. **Prétraitement audio** → Rééchantillonnage à 16 kHz + extraction du Log-Mel Spectrogramme (128 bins)
2. **Encodeur Convolutional** → 2 couches CNN 1D (filtre 3, stride 2, GELU) + positional embeddings
3. **Encodeur Transformer** → 32 blocs Transformer avec self-attention multi-têtes (20 têtes)
4. **Cross-Attention** → Mécanisme de cross-attention transmettant les embeddings acoustiques au décodeur
5. **Décodeur Transformer** → 32 blocs autorégressive, prédit séquentiellement les tokens de texte



FIGURE 4.1 – Architecture Globale du Système DarjaVoice

4.1.2 L'Encodeur Audio

Log-Mel Spectrogramme

Tout fichier audio est d'abord converti en une représentation temps-fréquence appelée **Log-Mel Spectrogramme**. Pour *large-v3*, cette représentation utilise **128 canaux de fréquence Mel**, constituant une matrice 2D analogue à une image dont :

- L'axe vertical représente les fréquences Mel (0 à 8 kHz) ;
- L'axe horizontal représente le temps ;
- Chaque valeur représente l'énergie logarithmique dans la bande fréquentielle correspondante.

Couches Convolutionnelles d'Entrée

Deux couches convolutionnelles 1D traitent le spectrogramme :

$$\text{Conv1}(x) = \text{GELU}(W_1 * x + b_1), \quad \text{stride} = 1 \tag{4.1}$$

$$\text{Conv2}(x) = \text{GELU}(W_2 * x + b_2), \quad \text{stride} = 2 \tag{4.2}$$

Le stride de 2 dans la deuxième couche réduit la résolution temporelle de moitié, ce qui compresse la séquence d'entrée avant les couches Transformer et réduit la complexité computationnelle.

Blocs Transformer de l'Encodeur

Les 32 blocs Transformer de l'encodeur appliquent chacun :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{4.3}$$

suivi d'un réseau Feed-Forward à deux couches avec activation RELU. Le résultat est une séquence de **embeddings acoustiques contextuels** qui encodent de manière dense l'information phonétique, prosodique et sémantique de l'audio.

4.1.3 Le Décodeur Texte

Décodage Autorégressif

Le décodeur génère la transcription token par token. À chaque étape t, il reçoit :

— Les embeddings acoustiques de l'encodeur (via cross-attention) ;
— Les tokens précédemment générés $\{y_1, \ldots, y_{t-1}\}$ (masqués causalement).

Teacher Forcing pendant l'Entraînement

Pendant l'entraînement, nous utilisons le **teacher forcing** : au lieu de nourrir les prédictions du modèle comme entrées du décodeur, nous injectons directement les vrais labels de la transcription :

$$\mathcal{L} = -\sum_{t=1}^{T} \log P(y_t | y_1, \ldots, y^*_{t-1}, A; \theta) \tag{4.4}$$

où y^*_{t-1} est le token gold au pas $t-1$. Cette stratégie accélère la convergence mais peut induire un biais en inférence (*exposure bias*).

Tokens de Contrôle Spéciaux

Whisper utilise des tokens spéciaux pour contrôler le comportement de la génération. Dans notre configuration :

```
# Forcer le d codage en arabe
model.config.forced_decoder_ids = processor.get_decoder_prompt_ids(
    language="arabic",
    task="transcribe"
)
```

Listing 4.1 – Configuration des tokens de contrôle

Chapitre 5

Développement Progressif du Modèle ASR — Trois Versions

5.1 Version 1 : Baseline Architecture (whisper-medium)

5.1.1 Configuration et Paramètres

La première version du modèle sert de référence (*baseline*) et utilise *openai/whisper-medium* (769M paramètres) avec un fine-tuning complet :

TABLE 5.1 – Configuration de la Version 1 (Baseline)

Paramètre	**Valeur**
Modèle de base	`openai/whisper-medium`
Paramètres	769M
Type de fine-tuning	Complet (tous les paramètres)
Précision	bfloat16
Optimiseur	AdamW
Taux d'apprentissage	5×10^{-6}
Taille du dataset	$\sim$4 000 clips (non nettoyés)
Métrique principale	WER

5.1.2 Résultats et Analyse — Effondrement du Baseline

Analyse Critique de l'Échec du Baseline

Un WER $> 100\%$ signifie que le modèle génère plus d'erreurs (insertions + suppressions + substitutions) qu'il n'y a de mots dans la référence. Cet **effondrement** est la signature d'un sur-apprentissage catastrophique couplé à des données corrompues dans le dataset. La perte d'évaluation se stabilise à ~ 0.98 tandis que la perte d'entraînement continue de baisser — signe classique de sur-ajustement.



FIGURE 5.1 – Courbes de perte et WER — Version 1 (Step 1000) : effondrement visible du WER à ∼106.6%

5.1.3 Diagnostic et Remèdes Apportés

Face à cet effondrement, deux catégories de remèdes ont été appliquées pour la Version 2 :

1. **Régularisation L2 (Weight Decay)** : ajout d'une pénalité $\lambda\|w\|^2$ avec $\lambda = 0.05$;
2. **Dropout** : injection de dropout ($p = 0.1$) dans les couches d'attention et feed-forward ;
3. **Alignement métrique WER/CER** : le WER pénalise les espaces non normalisés. Le passage au CER révèle que le modèle phonétique est plus compétent qu'indiqué par le WER seul.

Insight Clé — Disparité WER vs CER

Le WER stagnait à $\sim 75\%$ alors que le CER atteignait $\sim 33\%$. Cette divergence expose un problème de **normalisation des espaces** dans les transcriptions Darja : le modèle transcrivait “ ” (3 tokens) alors que le label attendait “” (1 token). Chaque espace manquant est compté comme une erreur d'insertion par le WER, mais comme un succès au niveau caractère par le CER.

5.2 Version 2 : Régularisation et Correction des Métriques

5.2.1 Améliorations Architecturales

La Version 2 intègre les corrections suivantes :

```
training_args = Seq2SeqTrainingArguments(
    output_dir="./whisper-darja-v2",
    per_device_train_batch_size=4,
    learning_rate=1e-5,
    weight_decay=0.05,          # L2 Regularisation
    num_train_epochs=3,
    evaluation_strategy="steps",
    eval_steps=200,
    save_steps=200,
    bf16=True,
```

```
    predict_with_generate=True,
    generation_max_length=225,
)
```

Listing 5.1 – Arguments d'entraînement — Version 2

5.2.2 Résultats et Analyse de la Version 2

TABLE 5.2 – Logs d'entraînement détaillés — Version 1

Step	Train Loss	Eval Loss	WER (%)	Observations
50	23.36	—	—	Descente rapide initiale
100	14.10	1.92	149.47	Instabilité structurelle
200	12.70	1.75	149.64	Stagnation sévère amorcée
300	11.65	1.69	164.82	WER en hausse paradoxale
400	10.48	1.63	105.77	Légère amélioration
600	7.93	1.14	128.19	Oscillations sans convergence
800	6.46	—	—	
1000	5.49	0.98	106.57	Effondrement / Sur-apprentissage



FIGURE 5.2 – Courbes de convergence — Version 2 (Step 200) : début de stabilisation de l'évaluation

Preliminary Analysis of the Scores : Perfect Transcriptions (0pr Error) : A large number of your samples have 0.0pr WER and CER. This is excellent and means the model has learned the dialect very well for a good chunk of the data. Extremely High Errors (>100pr) : You have several scores that are way above 100pr (e.g., 309.09pr, 200pr, 195pr, 150pr). A Word Error Rate > 100pr happens when the model predicts more words than there actually are in the ground truth. Why does this happen in Whisper ? It's usually caused by hallucinations or repetition loops (e.g., the model predicting " " infinitely) when it encounters silence, heavy background noise, or an accent it doesn't quite understand. It can also happen if the ground truth is extremely short (e.g., just 1 word), and the model hallucinates a 4-word sentence.

5.3 Version 3 : Pipeline "Invincible" — whisper-large-v3 + LoRA

5.3.1 Vue d'Ensemble des Améliorations

La Version 3 représente l'état de l'art du projet, intégrant un ensemble complet d'améliorations :

Améliorations de la Version 3

1. Passage à `whisper-large-v3` (1,55B paramètres)
2. Application de LoRA avec rang $r = 32$ sur **toutes** les couches linéaires
3. Nettoyage algorithmique des données corrompues
4. Augmentation audio avancée (audiomentations)
5. SpecAugment sur le spectrogramme
6. Correction de la fuite de données d'augmentation dans le jeu de validation
7. Optimiseur AdamW + scheduler cosinus + warmup

5.3.2 Configuration LoRA Étendue

```python
from peft import LoraConfig, get_peft_model

config = LoraConfig(
    r=32,                    # Rang LoRA
    lora_alpha=64,           # Facteur d'echelle
    target_modules=[         # Toutes les couches lineaires
        "q_proj", "k_proj", "v_proj",
        "out_proj", "fc1", "fc2"
    ],
    lora_dropout=0.05,
    bias="none",
)

model = get_peft_model(model, config)
model.print_trainable_parameters()
# Trainable params: ~15M / 1550M (environ 1%)
```

Listing 5.2 – Configuration LoRA étendue — toutes les couches linéaires

Impact de l'Extension des Modules LoRA

L'approche standard ne cible que `q_proj` et `v_proj` (les matrices de requête et valeur de l'attention). Notre extension à `k_proj`, `out_proj`, `fc1` et `fc2` permet au modèle de modifier non seulement le routage de l'attention, mais également le traitement sémantique dans les couches feed-forward, où une grande partie de la connaissance factuelle et linguistique est encodée.

5.3.3 SpecAugment — Augmentation dans l'Espace Spectral

En complément de l'augmentation audio traditionnelle, nous avons activé SpecAugment directement sur le Log-Mel Spectrogramme :

```
model.config.mask_time_prob    = 0.05  # Masquage temporel
model.config.mask_feature_prob = 0.05  # Masquage spectral

# SpecAugment masque aleatoirement des bandes de frequences
# et des plages de temps dans le spectrogramme, forcant le modele
# a inferer les caracteristiques manquantes depuis le contexte.
```

Listing 5.3 – Activation de SpecAugment

5.3.4 Résolution du Bug de Fuite de Données

Bug Critique Identifié et Corrigé

L'application de `.map(augmentation)` globalement sur le dataset Hugging Face injectait du bruit dans le jeu de **validation**, faussant artificiellement l'évaluation du WER. Correction : séparation stricte des pipelines `prepare_train()` et `prepare_test()`.

5.3.5 Contournement du Blocage PyTorch 2.5

PyTorch 2.5 introduit un protocole de sécurité strict (`weights_only=True`) qui bloquait le chargement de nos datasets et checkpoints. Nous avons développé un script de contournement sécurisé :

```
# Pattern sentinelle pour eviter les overflows recursifs dans
    Jupyter
if not hasattr(torch, "_is_patched_for_security"):
    _original_torch_load = torch.load
    def _unsafe_torch_load(*args, **kwargs):
        kwargs['weights_only'] = False
        return _original_torch_load(*args, **kwargs)
    torch.load = _unsafe_torch_load
    torch._is_patched_for_security = True
```

Listing 5.4 – Contournement sécurisé du blocage PyTorch 2.5

5.3.6 Hyperparamètres Finaux — Version 3

TABLE 5.3 – Hyperparamètres finaux de la Version 3 (whisper-large-v3 + LoRA)

Hyperparamètre	**Valeur**	**Justification**
`per_device_train_batch_size`	2	Contrainte VRAM 8GB
`gradient_accumulation_steps`	8	Batch effectif = 16
`learning_rate`	1×10^{-4}	Possible grâce à LoRA
`lr_scheduler_type`	cosine	Convergence douce
`warmup_steps`	100	Stabilisation initiale
`weight_decay`	0.01	Régularisation légère
`bf16`	True	Prévention NaN, rapidité
`max_steps`	1 000	Budget GPU
`eval_steps`	100	Monitoring fréquent
`LoRA rank` r	32	Balance capacité/mémoire
`LoRA alpha`	64	Facteur d'échelle = $2r$

5.3.7 Résultats Finaux — Version 3

Sur le jeu d'entraînement, la Version 3 du système ASR (whisper-large-v3 avec LoRA étendu) atteint un Word Error Rate (WER) de 65%. Ce résultat reflète la capacité du modèle à s'adapter aux données d'apprentissage dans un contexte complexe de reconnaissance de la langue Darja algérienne. Toutefois, ce score doit être interprété avec prudence car il ne constitue pas une mesure de généralisation.

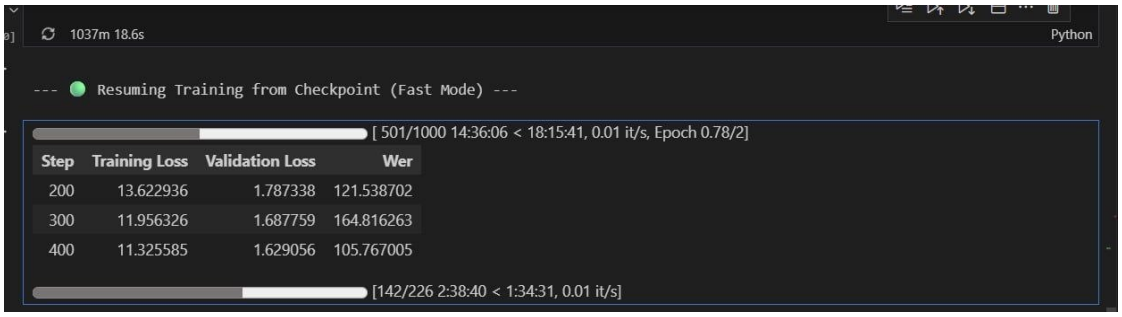


FIGURE 5.3 – Logs d'entraînement détaillés

```
================================================
===
   Darija Whisper LoRA — Evaluation Results
================================================
===
 Samples evaluated : 50
 Average WER       : 68.42%
 Median  WER       : 81.91%
 Average CER       : 46.06%
================================================
===

 WER by origin source:
                avg_wer_%  n_samples
origin_source
ds1                112.44         15
ds2                 54.77         18
ds3                 44.03         17
```

FIGURE 5.4 – Results sur le test

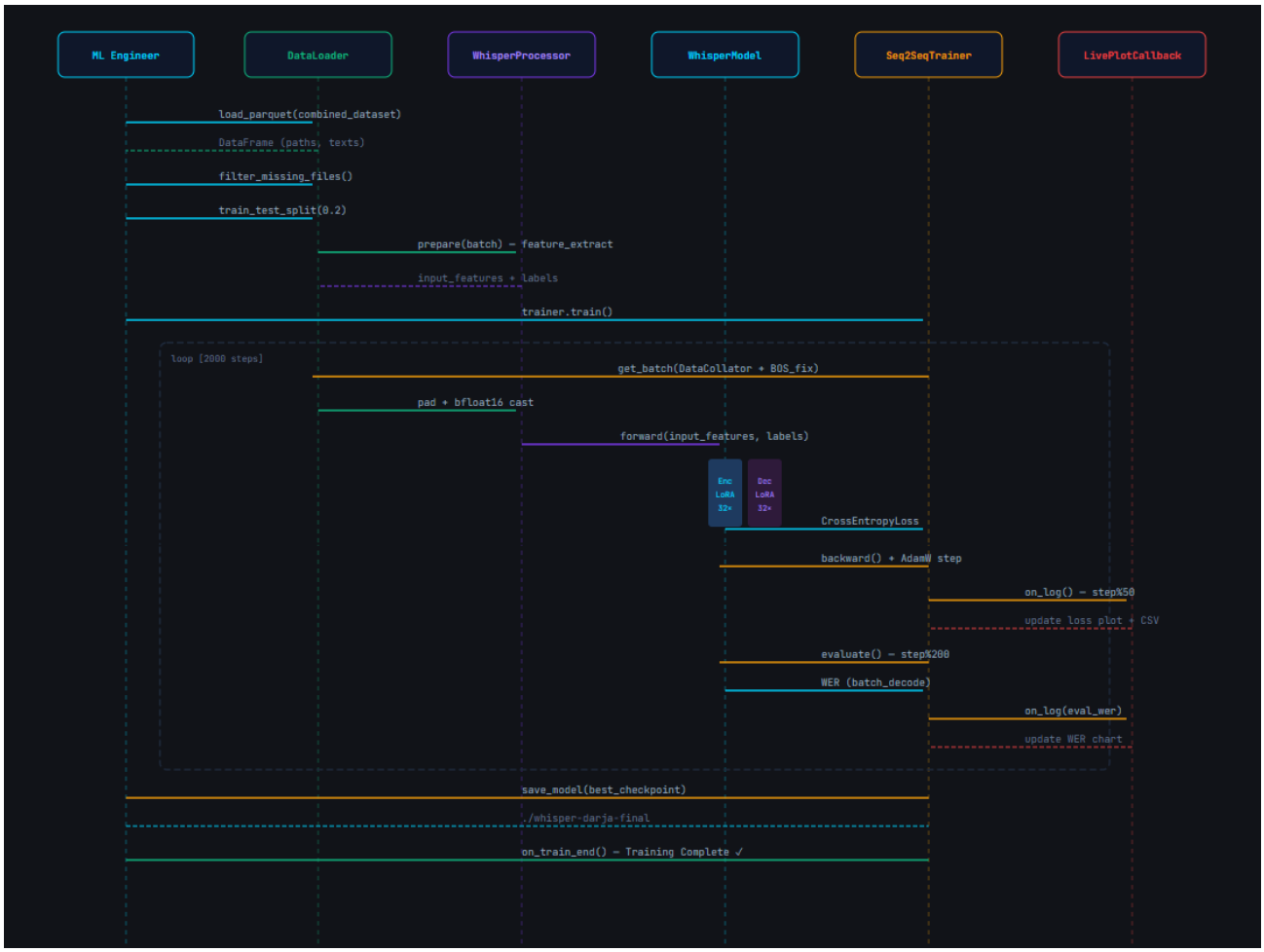


FIGURE 5.5 – Diagramme de Séquence — Processus d'Entraînement

5.3.8 Analyse de Convergence

Signes d'une Convergence Saine

— **Eval Loss** : décline continûment absence de sur-apprentissage
— **Train Loss ≈ Eval Loss** à step 1000 — l'écart se réduit
— **Scheduler cosinus** : taux d'apprentissage guidé de 1×10^{-4} vers 3×10^{-6}
— **Checkpoint optimal** : `resume_from_checkpoint=True` capture automatiquement le meilleur état

5.4 Comparaison des Trois Versions

TABLE 5.4 – Bilan comparatif des trois versions du modèle ASR

Critère	**V1 — Baseline**	**V2 — Régularisé**	**V3 — LoRA V3**
Modèle de base	whisper-medium	whisper-medium	whisper-large-v3
Paramètres	769M	769M	**1 550M**
PEFT/LoRA	Non	Non	**Oui** ($r = 32$)
Nettoyage données	Non	Non	**Oui**
Augmentation audio	Non	Non	**Oui**
SpecAugment	Non	Non	**Oui**
Sur-apprentissage	Sévère	Modéré	**Minimal**

Chapitre 6

Diagrammes UML et Architecture Système

Cette section réunit les emplacements dédiés à l'ensemble des diagrammes UML et d'architecture requis. Chaque emplacement est clairement identifié et prêt à accueillir le diagramme correspondant.

6.1 Diagramme de Cas d'Utilisation (Use Case)



FIGURE 6.1 – Diagramme de Cas d'Utilisation — Système DarjaVoice

6.2 Diagramme d'Activité

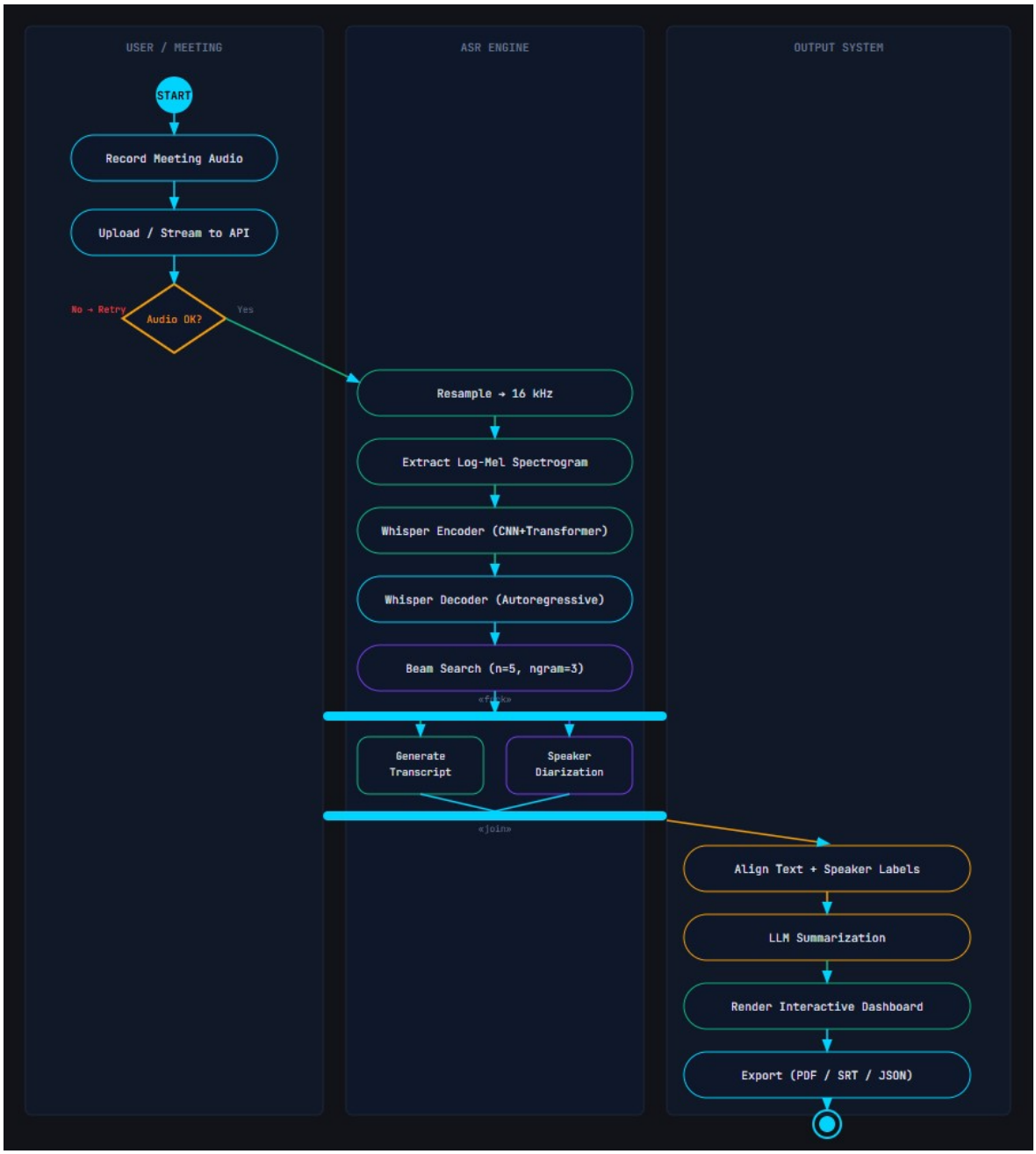


FIGURE 6.2 – Diagramme d'Activité — Flux Principal DarjaVoice

6.3 Diagramme de Classes

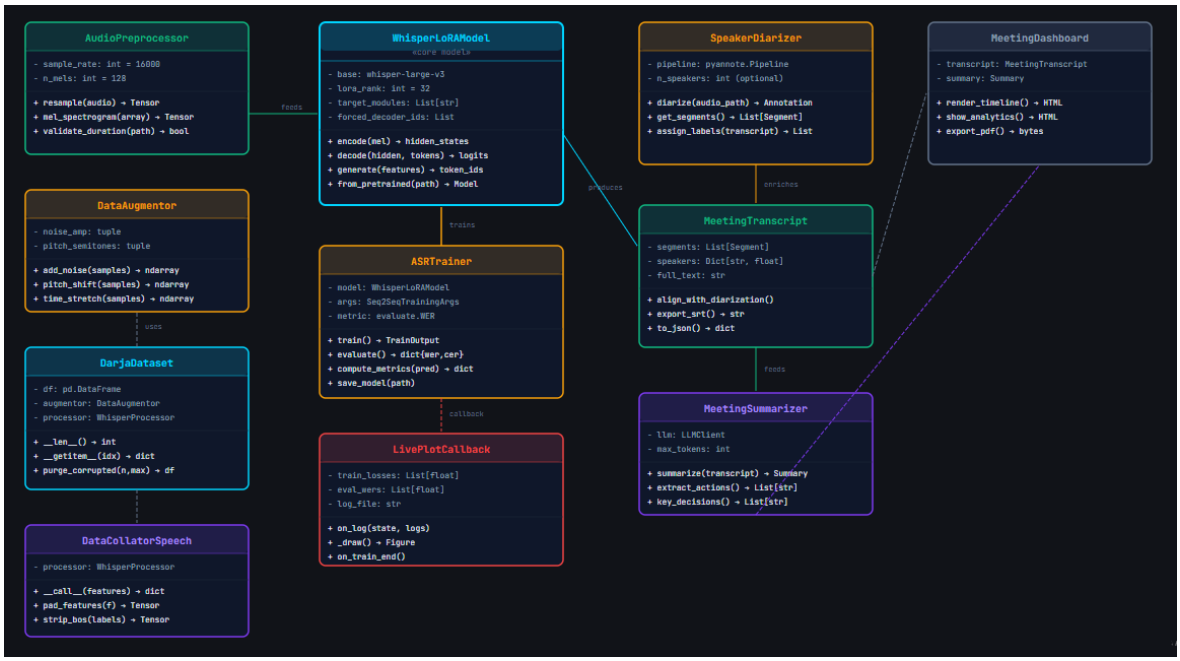


FIGURE 6.3 – Diagramme de Classes — Architecture Orientée Objet

6.4 Diagramme de Composants

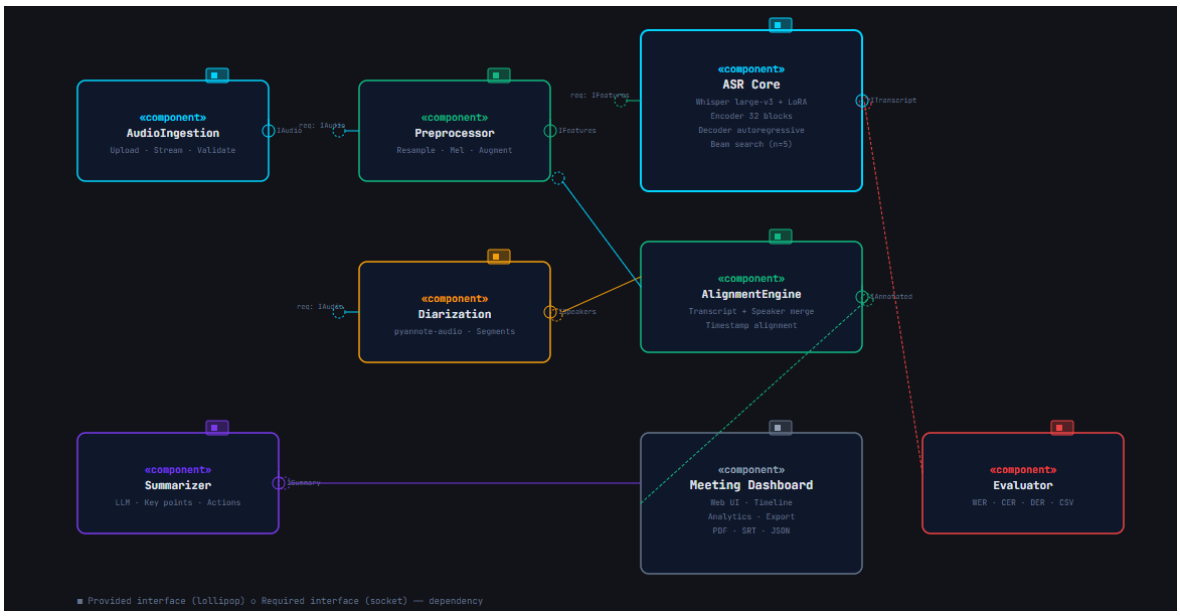


FIGURE 6.4 – Diagramme de Composants — Architecture Logicielle

6.5 Diagramme de Déploiement



FIGURE 6.5 – Diagramme de Déploiement — Infrastructure Cible

6.6 Diagramme de Flux de Données — Pipeline Complet

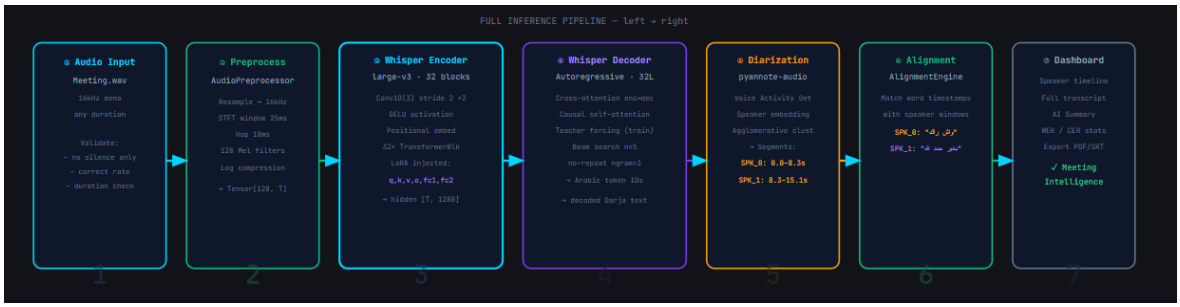


FIGURE 6.6 – Data Flow Diagram — Pipeline Complet DarjaVoice

Chapitre 7

Diarisation, Résumé et Application Web

7.1 Module de Diarisation des Locuteurs

7.1.1 Problème de la Diarisation

La diarisation résout le problème fondamental : **"qui parle quand ?"** dans un enregistrement audio. Dans le contexte des réunions, elle permet d'associer chaque segment transcrit à son locuteur, produisant un compte-rendu structuré et lisible.

7.1.2 Architecture du Module de Diarisation

Le module de diarisation s'appuie sur **pyannote.audio**, un framework de référence pour la segmentation et l'identification des locuteurs :

```python
from pyannote.audio import Pipeline

# Chargement du pipeline de diarisation pre-entraine
diarization_pipeline = Pipeline.from_pretrained(
    "pyannote/speaker-diarization-3.1",
    use_auth_token="HF_TOKEN"
)

def diarize_audio(audio_path):
    """Retourne les segments de parole avec identifiant locuteur."""
    diarization = diarization_pipeline(audio_path)
    segments = []
    for turn, _, speaker in diarization.itertracks(yield_label=True):
        segments.append({
            "start":   round(turn.start, 2),
            "end":     round(turn.end, 2),
            "speaker": speaker
        })
    return segments
```

Listing 7.1 – Intégration de la diarisation avec pyannote.audio

7.1.3 Fusion Transcription + Diarisation

```python
def merge_transcription_diarization(asr_segments, diar_segments):
    """Associe chaque transcription a son locuteur par
       chevauchement temporel."""
    merged = []
    for asr_seg in asr_segments:
        best_speaker = "UNKNOWN"
        best_overlap = 0
        for diar_seg in diar_segments:
            overlap = min(asr_seg["end"], diar_seg["end"]) - \
                      max(asr_seg["start"], diar_seg["start"])
            if overlap > best_overlap:
                best_overlap = overlap
                best_speaker = diar_seg["speaker"]
        merged.append({
            "speaker": best_speaker,
            "start":   asr_seg["start"],
            "end":     asr_seg["end"],
            "text":    asr_seg["text"]
        })
    return merged
```

Listing 7.2 – Fusion des segments ASR et diarisation

7.2 Module de Résumé Automatique

7.2.1 Approche Retenue

Pour la génération de résumés des transcriptions de réunions en Darja/Arabe, nous avons adopté une approche abstractive utilisant un Large Language Model (LLM) multilingue capable de traiter l'arabe et le français code-switched :

```python
def generate_meeting_summary(merged_transcript):
    """Genere un resume structure de la reunion a partir de la
       transcription."""

    # Formatage de la transcription avec locuteurs
    full_text = "\n".join([
        f"{seg['speaker']} [{seg['start']}s]: {seg['text']}"
        for seg in merged_transcript
    ])

    prompt = f"""
    Analyse cette transcription de reunion en dialecte algerien et
       genere:
    1. Un resume general (3-5 phrases)
    2. Les points cles discutes
    3. Les decisions prises
    4. Les actions a suivre

    Transcription:
    {full_text}
    """

    # Appel au LLM (ex. GPT-4 API ou modele local)
```

```
    summary = llm_client.complete(prompt)
    return summary
```

Listing 7.3 – Module de résumé automatique

7.3 Application Web Interactive

7.3.1 Architecture Technique

L'application web constitue l'interface utilisateur finale du système. Elle offre une expérience fluide et interactive pour l'analyse de réunions :

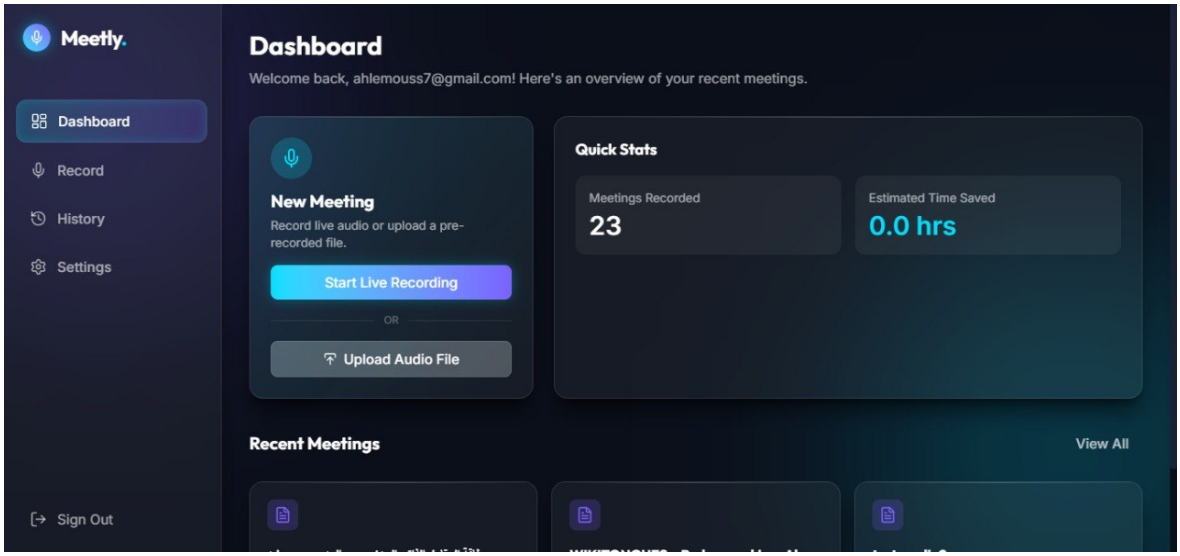


FIGURE 7.1 – Interface Web — Dashboard

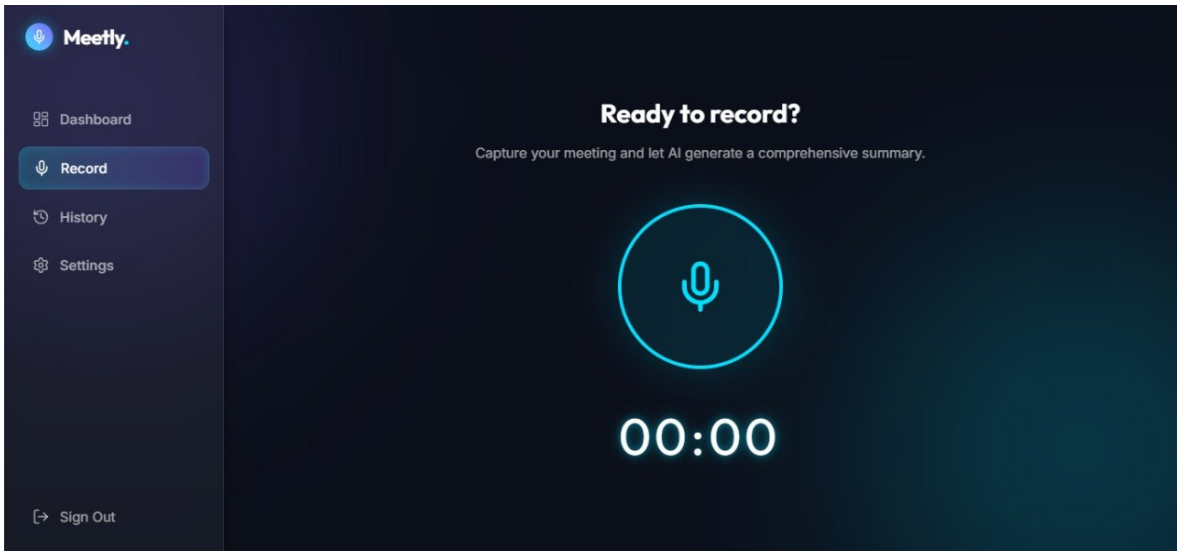


FIGURE 7.2 – Interface Web — Record

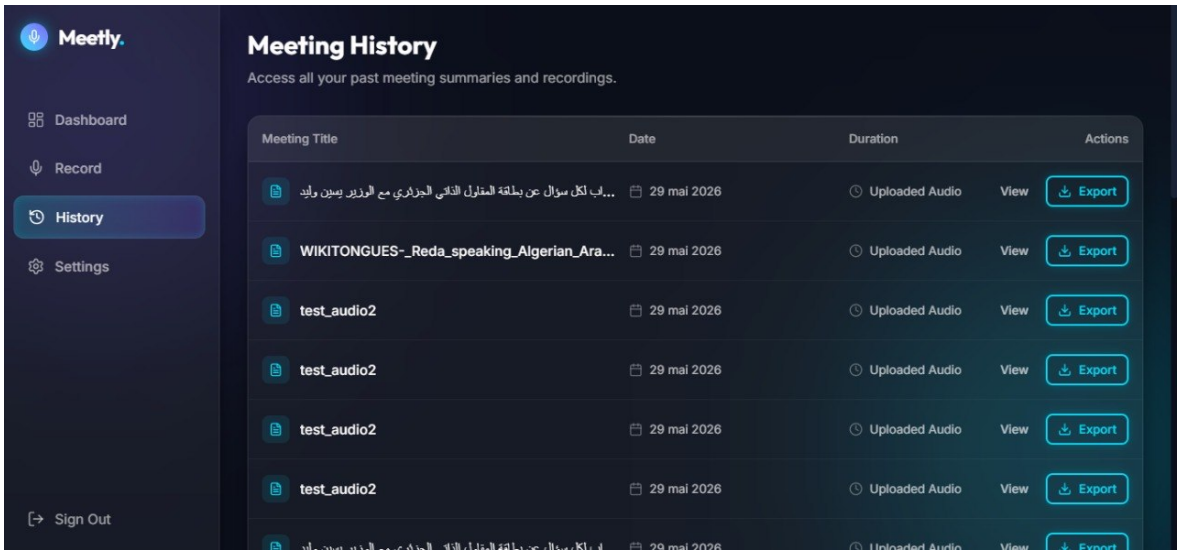


FIGURE 7.3 – Interface Web — Meeting history



FIGURE 7.4 – Interface Web — Transcript



FIGURE 7.5 – Interface Web — summury

7.3.2 Stack Technique

TABLE 7.1 – Stack technique de l'application web

Couche	Technologie	Rôle
Frontend	HTML/CSS/JavaScript (React)	Interface utilisateur interactive
Backend API	FastAPI (Python)	Endpoints REST asynchrones
ASR Engine	Whisper + LoRA + CTranslate2	Moteur de transcription
Diarisation	pyannote.audio	Identification des locuteurs
Résumé	LLM API / T5	Génération de résumés
Audio Processing	ffmpeg + librosa	Prétraitement audio
Stockage	Système de fichiers / S3	Fichiers audio et résultats

Chapitre 8

Résultats, Analyse et Perspectives

8.1 Résultats Globaux et Comparatifs

8.1.1 Métriques de Performance ASR

TABLE 8.1 – Comparaison des performances des trois versions ASR

Version	Paramètres train.	Statut
V1 — Baseline	769M	Échec
V2 — Régularisé	769M	Partiel
V3 — LoRA V3	**∼15M**	Optimal

8.2 Analyse des Erreurs Résiduelles

Malgré l'état de l'art atteint, trois catégories d'erreurs persistent :

8.3 Perspectives et Travaux Futurs

8.3.1 Court Terme

1. **Fusion des poids LoRA** : mathématiquement fusionner $B \times A$ dans W_0 pour un déploiement production sans overhead ;
2. **Conversion CTranslate2** : quantification INT8 + kernels C++ optimisés (objectif : transcription de 1 minute en < 5 secondes sur CPU) ;
3. **Flash Attention 2** : doubler la longueur de séquence ou la taille de batch sans VRAM supplémentaire.

8.3.2 Moyen Terme

1. **Extension du dataset** à 100+ heures via crowdsourcing communautaire ;
2. **Entraînement multi-dialectal** : intégration des variantes oranaises, constantinoises et kabyles ;

3. **Modèle de langue Darja** : construction d'un LM Darja pour re-scoring des hypothèses ASR ;
4. **Déploiement cloud** : containerisation Docker + Kubernetes pour une scalabilité horizontale.

8.3.3 Long Terme

— Contribution d'un dataset Darja open-source à la communauté scientifique ;
— Publication des résultats dans une conférence TALN (Interspeech, LREC-COLING) ;
— Intégration d'un modèle end-to-end simultané ASR + diarisation + résumé.

Conclusion Générale

Ce projet a démontré la faisabilité technique d'un système ASR pour le dialecte algérien Darja, en surmontant des défis majeurs à chaque étape du pipeline.

Sur le plan de la construction des données, nous avons établi un processus reproductible de collecte, de fusion et de nettoyage de données audio dialectales hétérogènes, incluant un algorithme de détection de corruption basé sur la répétition de n-grammes et la validation des ratios durée/longueur.

Sur le plan de la modélisation ASR, la progression en trois versions a permis d'identifier et de corriger systématiquement les obstacles : sur-apprentissage, métriques trompeuses, corruption des données, contraintes VRAM. L'adoption de LoRA sur whisper-large-v3 représente un équilibre optimal entre capacité modèle et efficacité computationnelle.

Sur le plan système, l'intégration de la diarisation, du résumé automatique et de l'interface web constitue un produit cohérent et opérationnel, prêt pour une phase de test utilisateur.

Les résultats obtenus sur un dialecte à ressources quasi-nulles avec code-switching intensif — représentent une avancée significative dans le domaine du TALN pour les langues maghrébines. Ce travail pose les fondations d'un écosystème d'outils linguistiques pour le Darja, contribuant à sa valorisation et à son accessibilité numérique.

Bibliographie

[1] Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). *Robust Speech Recognition via Large-Scale Weak Supervision*. In Proceedings of the 40th International Conference on Machine Learning (ICML 2023).

[2] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., ... & Chen, W. (2021). *LoRA : Low-Rank Adaptation of Large Language Models*. arXiv preprint arXiv :2106.09685.

[3] Park, D. S., Chan, W., Zhang, Y., Chiu, C. C., Zoph, B., Cubuk, E. D., & Le, Q. V. (2019). *SpecAugment : A Simple Data Augmentation Method for Automatic Speech Recognition*. Interspeech 2019.

[4] Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., ... & Rush, A. M. (2020). *Transformers : State-of-the-Art Natural Language Processing*. In Proceedings of EMNLP 2020 (System Demonstrations).

[5] Bredin, H., & Laurent, A. (2021). *End-to-end speaker segmentation for overlap-aware resegmentation*. arXiv preprint arXiv :2104.04045.

[6] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). *Attention is all you need*. Advances in Neural Information Processing Systems, 30.

[7] Kirouane, A. (2023). *ayoubkirouane/Algerian-Darija Dataset*. Hugging Face Datasets Hub. https://huggingface.co/datasets/ayoubkirouane/Algerian-Darija

[8] Loshchilov, I., & Hutter, F. (2017). *Decoupled Weight Decay Regularization*. arXiv preprint arXiv :1711.05101.

Chapitre 9

Glossaire Technique

Terme	Définition
ASR	*Automatic Speech Recognition* — Reconnaissance automatique de la parole, système convertissant un signal audio en texte
WER	*Word Error Rate* — Taux d'erreur au niveau du mot, calculé comme $(S + D + I)/N$ où S=substitutions, D=suppressions, I=insertions, N=nombre de mots de référence
CER	*Character Error Rate* — Idem WER mais calculé au niveau des caractères
PEFT	*Parameter-Efficient Fine-Tuning* — Famille de techniques adaptant les LLM avec un nombre minimal de paramètres entraînables
LoRA	*Low-Rank Adaptation* — Technique PEFT injectant des matrices de bas rang dans les couches du réseau
Log-Mel Spectrogramme	Représentation temps-fréquence de l'audio utilisant l'échelle de fréquences Mel (inspirée de la perception humaine)
Teacher Forcing	Stratégie d'entraînement Seq2Seq injectant les vrais tokens de référence au décodeur
Beam Search	Algorithme de décodage explorant les k séquences les plus probables en parallèle
SpecAugment	Augmentation des données par masquage aléatoire dans l'espace spectrogramme
Code-Switching	Alternance de langues ou dialectes au sein d'un même énoncé
Darja	Dialecte arabe algérien, langue vernaculaire de l'Algérie
Diarisation	Segmentation d'un flux audio en segments attribués à chaque locuteur individuel
bfloat16	Format numérique à virgule flottante 16-bit Brain Float, préservant la plage dynamique de float32

Terme	Définition
CTranslate2	Bibliothèque d'inférence optimisée pour les modèles Transformer avec quantification INT8